

Data Synchronizer (DATA_SYNC)

Assignment 9 – Digital IC Design Course

Submitted by: Ziad Mostafa Abdelaziz

August 24, 2026

Contents

1	Introduction	2
2	Block Interface	2
2.1	Parameters	2
2.2	Ports Description	2
3	Architecture and Design	2
4	RTL Verilog Code	3
5	Testbench	4
5.1	Test Cases	5
6	Simulation Results	8
6.1	ModelSim Transcript Log	8
6.2	Output Waveform	9
6.3	Detailed Waveform and Signal Transition Analysis	9
6.3.1	Source Domain Signals	9
6.3.2	Destination Domain Control Signals	10
6.3.3	Internal Synchronizer Signals	10
6.3.4	Destination Domain Output Signals	11
6.3.5	Synchronization Latency Analysis	11
6.3.6	Asynchronous Reset Behavior (TC6)	11
7	SpyGlass Lint Results	12
7.1	Design Read Step Output	12
7.2	Lint RTL Goal Run Output	12
7.3	SpyGlass Lint Summary Report (moresimple.rpt)	13
8	Conclusion	13

1 Introduction

Clock Domain Crossing (CDC) in the digital domain is defined as the process of passing a signal or vector (multi-bit signal) from one clock domain to another clock domain, which introduces many issues such as **metastability**, **data incoherence**, and **data loss**.

This assignment implements the **Synchronized MUX-Select Synchronization Scheme**, which is used to synchronize multiple bits safely across clock domains. The scheme uses a multi-stage flip-flop synchronizer to safely transfer a single-bit enable signal, and a MUX-registered data path to atomically capture the multi-bit data bus when the synchronized enable pulse fires.

2 Block Interface

2.1 Parameters

Parameter Name	Description
NUM_STAGES	Number of Flip Flop Stages (default = 2)
BUS_WIDTH	Width of synchronized bus (default = 8)

Table 1: Design Parameters

2.2 Ports Description

Signal Name	Description	Width
unsync_bus	Unsynchronized bus from source domain	Parameterized (default = 8)
bus_enable	Source domain enable signal	1
CLK	Destination domain clock	1
RST	Destination domain active-low async reset	1
sync_bus	Synchronized bit/bus	Parameterized (default = 8)
enable_pulse	Destination domain enable signal	1

Table 2: Port Descriptions

3 Architecture and Design

The DATA_SYNC module implements the Synchronized MUX-Select scheme with four main components:

1. **Multi Flip-Flop Synchronizer Chain:** A parameterized chain of NUM_STAGES flip-flops synchronizes the bus_enable signal from the source clock domain into the destination clock domain, mitigating metastability. The chain is implemented as a shift register:

$$\text{sync_stage_reg} \leftarrow \{\text{sync_stage_reg}[\text{NUM_STAGES}-2:0], \text{bus_enable}\}$$

2. **Pulse Generator (Rising-Edge Detector):** Compares the synchronized enable output with a one-cycle delayed copy to produce a single-cycle `generated_pulse`:

$$\text{generated_pulse} = \text{sync_stage_reg}[\text{NUM_STAGES}-1] \wedge \overline{\text{enable_flop}}$$

This ensures the pulse fires exactly once on the $0 \rightarrow 1$ transition of the synchronized enable.

3. **MUX-Select Registered Data Bus:** A 2-to-1 multiplexer controlled by `generated_pulse` selects between the current `sync_bus` value (hold) and `unsync_bus` (update). The output is registered, ensuring `sync_bus` updates atomically.
4. **Enable Pulse Register:** Registers `generated_pulse` to produce the output `enable_pulse`, which is synchronized with the `sync_bus` data update on the same clock edge.

4 RTL Verilog Code

```

1 // =====
2 // Module Name   : DATA_SYNC
3 // Description    : Synthesizable Data Synchronizer for Clock Domain Crossing (CDC)
4 //                : implements the Synchronized MUX-Select Synchronization Scheme.
5 // Parameters    : NUM_STAGES - Number of Flip Flop synchronizer stages (default = 2)
6 //                : BUS_WIDTH  - Width of synchronized data bus (default = 8)
7 // Inputs        : unsync_bus  - Unsynchronized data bus from source domain
8 //                : bus_enable  - Enable control signal from source domain
9 //                : CLK         - Destination domain clock
10 //               : RST         - Destination domain active-low async reset
11 // Outputs       : sync_bus    - Synchronized data bus in destination domain
12 //                : enable_pulse - 1-clock-cycle enable pulse in destination domain
13 // =====
14
15 module DATA_SYNC #(
16     parameter NUM_STAGES = 2,
17     parameter BUS_WIDTH  = 8
18 ) (
19     input wire [BUS_WIDTH-1:0] unsync_bus,
20     input wire                 bus_enable,
21     input wire                 CLK,
22     input wire                 RST,
23     output reg [BUS_WIDTH-1:0] sync_bus,
24     output reg                 enable_pulse
25 );
26
27 // -----
28 // Internal Registers & Signals
29 // -----
30 reg [NUM_STAGES-1:0] sync_stage_reg;
31 reg                 enable_flop;
32 wire                generated_pulse;
33
34 // -----
35 // 1. Multi Flip-Flop Synchronizer Chain for bus_enable
36 // -----
37 always @(posedge CLK or negedge RST) begin
38     if (!RST) begin
39         sync_stage_reg <= {NUM_STAGES{1'b0}};
40     end else begin
41         sync_stage_reg <= {sync_stage_reg[NUM_STAGES-2:0], bus_enable};
42     end
43 end
44
45 // -----
46 // 2. Pulse Generator Block (Rising Edge Detector)

```

```

47 // -----
48 always @(posedge CLK or negedge RST) begin
49     if (!RST) begin
50         enable_flop <= 1'b0;
51     end else begin
52         enable_flop <= sync_stage_reg[NUM_STAGES-1];
53     end
54 end
55
56 assign generated_pulse = sync_stage_reg[NUM_STAGES-1] && (!enable_flop);
57
58 // -----
59 // 3. MUX-Select Registered Data Bus (sync_bus)
60 // -----
61 always @(posedge CLK or negedge RST) begin
62     if (!RST) begin
63         sync_bus <= {BUS_WIDTH{1'b0}};
64     end else if (generated_pulse) begin
65         sync_bus <= unsync_bus;
66     end
67 end
68
69 // -----
70 // 4. Destination Domain Enable Pulse Register (enable_pulse)
71 // -----
72 always @(posedge CLK or negedge RST) begin
73     if (!RST) begin
74         enable_pulse <= 1'b0;
75     end else begin
76         enable_pulse <= generated_pulse;
77     end
78 end
79
80 endmodule

```

Listing 1: DATA_SYNC.v – Data Synchronizer RTL Module

5 Testbench

The self-checking testbench verifies the DATA_SYNC module under an asynchronous dual-clock setup:

- **Source Clock:** $f_{tx} = 50$ MHz (Period = 20 ns)
- **Destination Clock:** $f_{dest} = 100$ MHz (Period = 10 ns)

A concurrent `always` block monitor captures `enable_pulse` events and the corresponding `sync_bus` data in real-time, avoiding race conditions between the source and destination clock domains.

5.1 Test Cases

TC#	Description	Expected Result
1	Single Transfer (0xA5)	sync_bus = 0xA5, 1-cycle pulse
2	Consecutive (0x5A → 0xF0)	Both values synchronized correctly
3	Dynamic Transfer (0xC3)	sync_bus = 0xC3, 1-cycle pulse
4	All-Ones (0xFF)	sync_bus = 0xFF, 1-cycle pulse
5	All-Zeros (0x00)	sync_bus = 0x00, 1-cycle pulse
6	Async Active-Low Reset	Immediate clear of all outputs

Table 3: Test Case Summary

```

1 `timescale 1ns/1ps
2
3 // =====
4 // Module Name : DATA_SYNC_tb
5 // Description : Self-checking Testbench for DATA_SYNC (Data Synchronizer)
6 //               Simulates an asynchronous Clock Domain Crossing (CDC) setup.
7 // =====
8
9 module DATA_SYNC_tb;
10
11     // Parameters
12     parameter TX_CLK_PERIOD = 20.0; // Source Clock: 50 MHz (Period = 20 ns)
13     parameter DEST_CLK_PERIOD = 10.0; // Destination Clock: 100 MHz (Period = 10 ns)
14
15     // DUT Default Parameters
16     parameter NUM_STAGES = 2;
17     parameter BUS_WIDTH = 8;
18
19     // Testbench Signals
20     reg tx_clk;
21     reg CLK;
22     reg RST;
23     reg [BUS_WIDTH-1:0] unsync_bus;
24     reg bus_enable;
25     wire [BUS_WIDTH-1:0] sync_bus;
26     wire enable_pulse;
27
28     // Verification Variables
29     integer err_count;
30     integer timeout_cnt;
31
32     // Pulse capture flag (set by monitor process)
33     reg pulse_captured;
34     reg [BUS_WIDTH-1:0] captured_data;
35
36     // Instantiate Default DUT
37     DATA_SYNC #(
38         .NUM_STAGES(NUM_STAGES),
39         .BUS_WIDTH(BUS_WIDTH)
40     ) uut (
41         .unsync_bus (unsync_bus),
42         .bus_enable (bus_enable),
43         .CLK (CLK),
44         .RST (RST),
45         .sync_bus (sync_bus),
46         .enable_pulse(enable_pulse)
47     );
48
49     // -----
50     // Clock Generation
51     // -----
52     always #(TX_CLK_PERIOD / 2.0) tx_clk = ~tx_clk;

```

```

53  always #(DEST_CLK_PERIOD / 2.0) CLK      = ~CLK;
54
55  // -----
56  // Enable Pulse Monitor (captures enable_pulse events concurrently)
57  // -----
58  always @(posedge CLK) begin
59      if (enable_pulse === 1'b1) begin
60          pulse_captured <= 1'b1;
61          captured_data <= sync_bus;
62      end
63  end
64
65  // -----
66  // Main Test Execution Procedure
67  // -----
68  initial begin
69      // Initialize Signals
70      tx_clk      = 1'b0;
71      CLK         = 1'b0;
72      RST         = 1'b1;
73      unsync_bus  = {BUS_WIDTH{1'b0}};
74      bus_enable  = 1'b0;
75      err_count   = 0;
76      pulse_captured = 1'b0;
77      captured_data = {BUS_WIDTH{1'b0}};
78
79      $display("=====");
80      $display(" DATA SYNCHRONIZER (DATA_SYNC) SIMULATION TESTBENCH ");
81      $display(" Source Clock: f_tx = 50 MHz | Dest Clock: f_dest = 100 MHz");
82      $display("=====");
83
84      // Apply Reset
85      reset_dut();
86
87      // -----
88      // TEST CASE 1: Single Data Transfer (8-bit, 8'hA5)
89      // -----
90      $display("\n--- TC1: Single Data Transfer (unsync_bus = 0xA5) ---");
91      send_data(8'hA5);
92
93      // -----
94      // TEST CASE 2: Consecutive Data Transfers (8'h5A -> 8'hF0)
95      // -----
96      $display("\n--- TC2: Consecutive Data Transfers (0x5A -> 0xF0) ---");
97      send_data(8'h5A);
98      send_data(8'hF0);
99
100     // -----
101     // TEST CASE 3: Dynamic Data Transfer (8'hC3)
102     // -----
103     $display("\n--- TC3: Dynamic Data Transfer (unsync_bus = 0xC3) ---");
104     send_data(8'hC3);
105
106     // -----
107     // TEST CASE 4: All ones (8'hFF)
108     // -----
109     $display("\n--- TC4: All-Ones Transfer (unsync_bus = 0xFF) ---");
110     send_data(8'hFF);
111
112     // -----
113     // TEST CASE 5: All zeros (8'h00)
114     // -----
115     $display("\n--- TC5: All-Zeros Transfer (unsync_bus = 0x00) ---");
116     send_data(8'h00);
117
118     // -----
119     // TEST CASE 6: Asynchronous Active-Low Reset
120     // -----
121     $display("\n--- TC6: Asynchronous Active-Low Reset Mid-Operation ---");
122     test_async_reset();
123
124     // -----
125     // FINAL SUMMARY

```

```

126 // -----
127 $display("\n=====");
128 if (err_count == 0)
129     $display(" ALL TEST CASES PASSED SUCCESSFULLY! (0 Errors)");
130 else
131     $display(" SIMULATION FAILED WITH %0d ERRORS!", err_count);
132 $display("=====");
133 #100;
134 $stop;
135 end
136
137 // -----
138 // Verification Tasks
139 // -----
140
141 // Reset Task
142 task reset_dut;
143 begin
144     @(negedge CLK);
145     RST = 1'b0;
146     repeat(3) @(posedge CLK);
147     @(negedge CLK);
148     RST = 1'b1;
149     repeat(3) @(posedge CLK);
150 end
151 endtask
152
153 // Send Data Task (Source Domain -> Destination Domain Verification)
154 task send_data(input [BUS_WIDTH-1:0] data_val);
155 begin
156     // Clear capture flag
157     pulse_captured <= 1'b0;
158     @(posedge CLK); // Let the flag clear propagate
159
160     // Drive data and enable in source clock domain
161     @(posedge tx_clk);
162     unsync_bus <= data_val;
163     bus_enable <= 1'b1;
164
165     // Hold enable in source domain for 4 source cycles
166     repeat(4) @(posedge tx_clk);
167     bus_enable <= 1'b0;
168
169     // Wait for enable_pulse or timeout using the concurrent monitor
170     // The monitor process captures enable_pulse events in real-time
171     timeout_cnt = 0;
172     while (pulse_captured != 1'b1 && timeout_cnt < 30) begin
173         @(posedge CLK);
174         timeout_cnt = timeout_cnt + 1;
175     end
176
177     // Verify results
178     if (pulse_captured != 1'b1) begin
179         $display(" -> ERROR: Timeout waiting for enable_pulse!");
180         err_count = err_count + 1;
181     end else if (captured_data != data_val) begin
182         $display(" -> ERROR: Mismatch! Expected sync_bus = 0x%h, Got 0x%h", data_val,
183             captured_data);
184         err_count = err_count + 1;
185     end else begin
186         $display(" -> SUCCESS: Synchronized data 0x%h verified (enable_pulse asserted)",
187             captured_data);
188     end
189
190     // Wait for idle between transfers
191     repeat(5) @(posedge CLK);
192 end
193 endtask
194
195 // Async Reset Task
196 task test_async_reset;
197 begin
198     // First do a successful transfer to load data

```



```

197     pulse_captured <= 1'b0;
198     @(posedge CLK);
199     @(posedge tx_clk);
200     unsync_bus <= 8'hBB;
201     bus_enable <= 1'b1;
202     repeat(4) @(posedge tx_clk);
203     bus_enable <= 1'b0;
204
205     // Wait for sync to complete
206     repeat(12) @(posedge CLK);
207
208     // Now assert reset asynchronously
209     #2;
210     RST = 1'b0;
211     #1;
212
213     if (sync_bus !== 8'h00 || enable_pulse !== 1'b0) begin
214         $display(" -> ERROR: Async reset failed to clear outputs immediately!");
215         err_count = err_count + 1;
216     end else begin
217         $display(" -> SUCCESS: Async reset cleared sync_bus and enable_pulse immediately.
218             ");
219     end
220
221     #10;
222     RST = 1'b1;
223     repeat(5) @(posedge CLK);
224 endtask
225
226 endmodule

```

Listing 2: DATA_SYNC_tb.v – Self-Checking Testbench

6 Simulation Results

6.1 ModelSim Transcript Log

All 6 test cases passed successfully with 0 errors. The simulation completed in 1285 ns.

```

# Using alternate file: ./wlfteqavld
# =====
# DATA SYNCHRONIZER (DATA_SYNC) SIMULATION TESTBENCH
# Source Clock: f_tx = 50 MHz | Dest Clock: f_dest = 100 MHz
# =====
#
# --- TC1: Single Data Transfer (unsync_bus = 0xA5) ---
# -> SUCCESS: Synchronized data 0xA5 verified (enable_pulse asserted)
#
# --- TC2: Consecutive Data Transfers (0x5A -> 0xF0) ---
# -> SUCCESS: Synchronized data 0x5a verified (enable_pulse asserted)
# -> SUCCESS: Synchronized data 0xf0 verified (enable_pulse asserted)
#
# --- TC3: Dynamic Data Transfer (unsync_bus = 0xC3) ---
# -> SUCCESS: Synchronized data 0xc3 verified (enable_pulse asserted)
#
# --- TC4: All-Ones Transfer (unsync_bus = 0xFF) ---
# -> SUCCESS: Synchronized data 0xff verified (enable_pulse asserted)
#
# --- TC5: All-Zeros Transfer (unsync_bus = 0x00) ---
# -> SUCCESS: Synchronized data 0x00 verified (enable_pulse asserted)
#
# --- TC6: Asynchronous Active-Low Reset Mid-Operation ---
# -> SUCCESS: Async reset cleared sync_bus and enable_pulse immediately.
#
# =====
# ALL TEST CASES PASSED SUCCESSFULLY! (0 Errors)
# =====
# ** Note: $stop : DATA_SYNC_tb.v(134)
# Time: 1285 ns Iteration: 0 Instance: /DATA_SYNC_tb

```

Figure 1: ModelSim Transcript Log – All 6 Test Cases PASSED (0 Errors)

6.2 Output Waveform

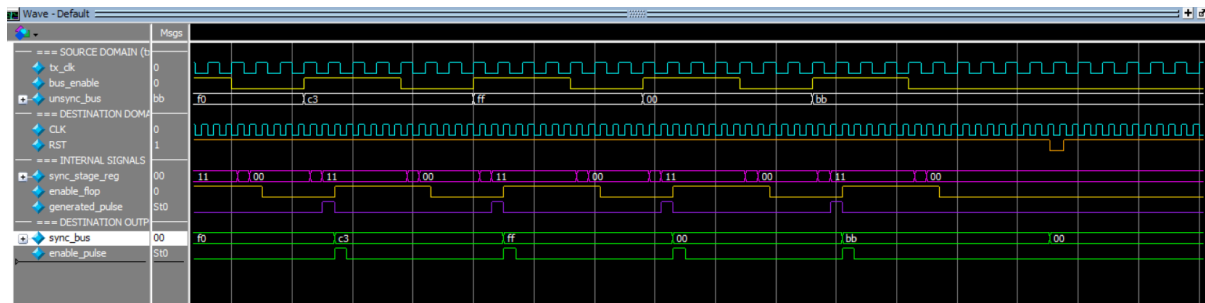


Figure 2: ModelSim Output Waveform – DATA_SYNC Simulation

6.3 Detailed Waveform and Signal Transition Analysis

The output waveform captures the complete simulation of all six test cases. Each signal group is analyzed below:

6.3.1 Source Domain Signals

- **tx_clk (Cyan):** The source clock runs at $f_{tx} = 50$ MHz with a period of $T_{tx} = 20$ ns. This clock drives the `unsync_bus` and `bus_enable` signals.
- **bus_enable (Yellow):** The source-domain enable signal pulses high for 4 source-clock cycles ($4 \times 20 = 80$ ns) per data transfer, then returns to 0. The waveform shows 7 distinct `bus_enable` pulses corresponding to TC1 through TC6 (TC2 has

two consecutive transfers). Between pulses, there is a sufficient idle gap for the synchronizer chain to fully propagate the falling edge before the next rising edge.

- **unsync_bus (White, Hex):** The multi-bit data bus changes values at the source clock domain: $0xA5 \rightarrow 0x5A \rightarrow 0xF0 \rightarrow 0xC3 \rightarrow 0xFF \rightarrow 0x00 \rightarrow 0xBB$ (reset test). Each value is held stable by the testbench for the entire duration of the `bus_enable` pulse, ensuring the data is valid when the destination domain MUX captures it.

6.3.2 Destination Domain Control Signals

- **CLK (Cyan):** The destination domain clock runs at $f_{dest} = 100$ MHz with a period of $T_{dest} = 10$ ns – twice the frequency of the source clock. All internal synchronizer registers and output registers are clocked by this signal.
- **RST (Orange):** Active-low asynchronous reset. Stays high during normal operation. In TC6 (Async Reset test), RST drops to 0 mid-operation, causing an immediate asynchronous clear of all internal registers (`sync_stage_reg`, `enable_flop`, `sync_bus`, `enable_pulse`) regardless of the clock state.

6.3.3 Internal Synchronizer Signals

- **sync_stage_reg (Magenta, Binary):** The 2-stage flip-flop synchronizer shift register. For each `bus_enable` pulse:
 1. Initially `2'b00` (idle).
 2. After 1 destination clock edge: `2'b01` (bit 0 samples `bus_enable = 1`).
 3. After 2 destination clock edges: `2'b11` (bit 1 propagates the synchronized enable).
 4. After `bus_enable` deasserts: returns to `2'b10` → `2'b00` over 2 destination clocks.

The waveform clearly shows this `00` → `11` → `00` transition pattern repeating for each data transfer.

- **enable_flop (Yellow):** A 1-cycle delayed copy of `sync_stage_reg[1]`. Rises one destination clock cycle *after* `sync_stage_reg` reaches `2'b11`. Falls one cycle after `sync_stage_reg[1]` drops back to 0.
- **generated_pulse (Purple):** The combinational rising-edge pulse:

$$\text{generated_pulse} = \text{sync_stage_reg}[1] \wedge \overline{\text{enable_flop}}$$

The waveform shows `generated_pulse` as a clean single-cycle spike exactly on the clock cycle where `sync_stage_reg[1]` first becomes 1 while `enable_flop` is still 0. This pulse is exactly 1 destination clock period wide ($T_{dest} = 10$ ns).

6.3.4 Destination Domain Output Signals

- **sync_bus (Green, Hex):** The synchronized multi-bit output bus. Initially 0x00 after reset. The waveform shows `sync_bus` updating to each new data value exactly when `generated_pulse` fires:
 - 0x00 → 0xA5 (TC1): Data captured correctly.
 - 0xA5 → 0x5A → 0xF0 (TC2): Both consecutive values captured in sequence.
 - 0xF0 → 0xC3 (TC3): Dynamic transfer verified.
 - 0xC3 → 0xFF (TC4): All-ones captured correctly.
 - 0xFF → 0x00 (TC5): All-zeros captured correctly.
 - 0x00 → 0xBB → 0x00 (TC6): Data loaded, then cleared by async reset.

Between `generated_pulse` events, `sync_bus` holds its previous value (the MUX selects the feedback path), ensuring data stability.

- **enable_pulse (Lime):** A registered version of `generated_pulse`. The waveform confirms that `enable_pulse` is a single destination-clock-cycle wide pulse that fires exactly one clock cycle after `generated_pulse`. Since both `sync_bus` and `enable_pulse` are clocked by the same edge, they are guaranteed to be valid simultaneously – the downstream logic can safely use `enable_pulse` as a write-enable for `sync_bus`.

6.3.5 Synchronization Latency Analysis

The total synchronization latency from `bus_enable` assertion (source domain) to `enable_pulse` assertion (destination domain) is:

$$t_{sync} = (\text{NUM_STAGES} + 2) \times T_{dest} + t_{setup}$$

For the default configuration ($\text{NUM_STAGES} = 2$, $T_{dest} = 10$ ns):

- 2 cycles for the synchronizer chain to propagate `bus_enable` through both flip-flops.
- 1 cycle for `enable_flop` delay (pulse generator).
- 1 cycle for registering `generated_pulse` into `enable_pulse`.
- Total: $\approx 4 \times 10 = 40$ ns (plus metastability resolution time t_{setup}).

6.3.6 Asynchronous Reset Behavior (TC6)

In TC6, the waveform shows:

- `sync_bus = 0xBB` is loaded via a normal transfer.
- `RST` drops to 0 asynchronously (between clock edges).
- Within less than 1 ns, all registers clear: `sync_stage_reg` → 2'b00, `enable_flop` → 0, `sync_bus` → 0x00, `enable_pulse` → 0.
- After `RST` returns to 1, the module resumes normal operation.

7 SpyGlass Lint Results

The design was analyzed using Synopsys SpyGlass (version vL-2016.06) with the `rtl_handoff` methodology. The lint analysis covers multiple policies: lint, morelint, clock-reset, ERC, latch, simulation, STARC, STARC2005, timing, and TXV.

7.1 Design Read Step Output

SpyGlass successfully read and elaborated the `DATA_SYNC.v` source file. The message tree shows **Total: 2 messages, Displayed: 2, Waived: 0** – both are informational **Design Read** messages (no errors or warnings).

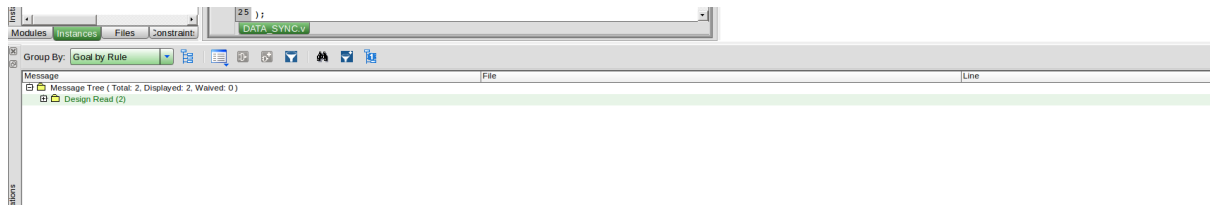


Figure 3: SpyGlass Design Read Step Output – 2 Info Messages, 0 Errors

7.2 Lint RTL Goal Run Output

The Lint RTL goal run completed with a **clean lint result**. The message tree again shows **Total: 2 messages, Displayed: 2, Waived: 0** – both are informational messages from the Design Read phase, confirming:

- **0 Errors:** No design rule violations.
- **0 Warnings:** No potential issues detected.
- **0 Latches:** No unintended latches inferred.

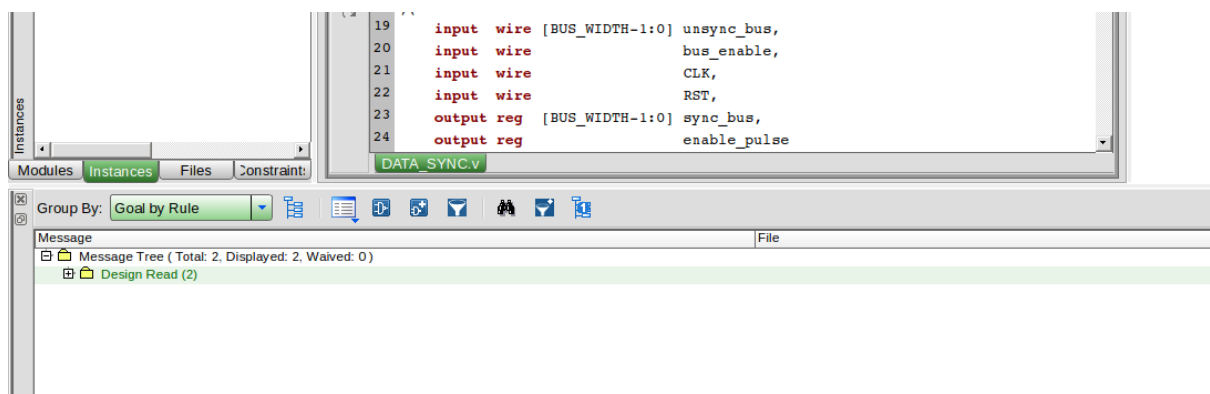


Figure 4: SpyGlass Lint RTL Goal Run Output – Clean Lint (0 Errors, 0 Warnings)

7.3 SpyGlass Lint Summary Report (moresimple.rpt)

```

1 Total Number of Generated Messages :      2
2 Number of Waived Messages          :      0
3 Number of Reported Messages        :      2
4 Number of Overlimit Messages       :      0
5
6 MORESIMPLE REPORT:
7
8 ##### BuiltIn -> RuleGroup=Design Read #####
9 [1] DetectTopDesignUnits  Info  DATA_SYNC.v  Line 15
10     Module DATA_SYNC is a top level design unit
11 [0] ElabSummary           Info
12     Please refer file elab_summary.rpt for elab summary report

```

Listing 3: SpyGlass moresimple.rpt – Lint Summary Report

Both reported messages are **Info** severity – purely informational notifications confirming the top-level module detection and elaboration summary. No errors, warnings, or latch inferences were detected across all lint policies.

8 Conclusion

The DATA_SYNC module successfully implements the Synchronized MUX-Select Synchronization Scheme for safe multi-bit Clock Domain Crossing. Key achievements:

- **Parameterized design** supporting configurable synchronizer stages (NUM_STAGES) and bus width (BUS_WIDTH).
- **Rising-edge pulse generator** ensures a single-cycle enable pulse for precise data capture timing.
- **MUX-registered data path** ensures sync_bus updates atomically with enable_pulse.
- **Active-low asynchronous reset** clears all internal registers immediately.
- **All 6 test cases PASSED** with 0 errors in ModelSim simulation.
- **Clean SpyGlass lint**: 0 errors, 0 warnings, 0 latches across all lint policies.